Deep Reinforcement Learning

Brief Introduction to Deep Learning

Prof. Dr. Sebastian Peitz

Chair of Safe Autonomous Systems, TU Dortmund

Summer term 2026

Content

Content

Content:

- The three main learning paradigms
- Supervised learning
  - The learning diagram
  - Generalization
  - Bias & variance
  - Cross-validation
- Deep neural networks
  - Artificial neural networks
  - Multi-layer perceptron
  - Training
  - Regularization
  - Neural network architectures
- Linear regression as a special case

Useful references:

- Pattern recognition and machine learning (Bishop 2006)
- Deep learning (Bengio et al. 2017)
- The elements of statistical learning (Hastie 2009)
- Learning from data: a short course (Abu-Mostafa, Magdon-Ismail, and Lin 2012)

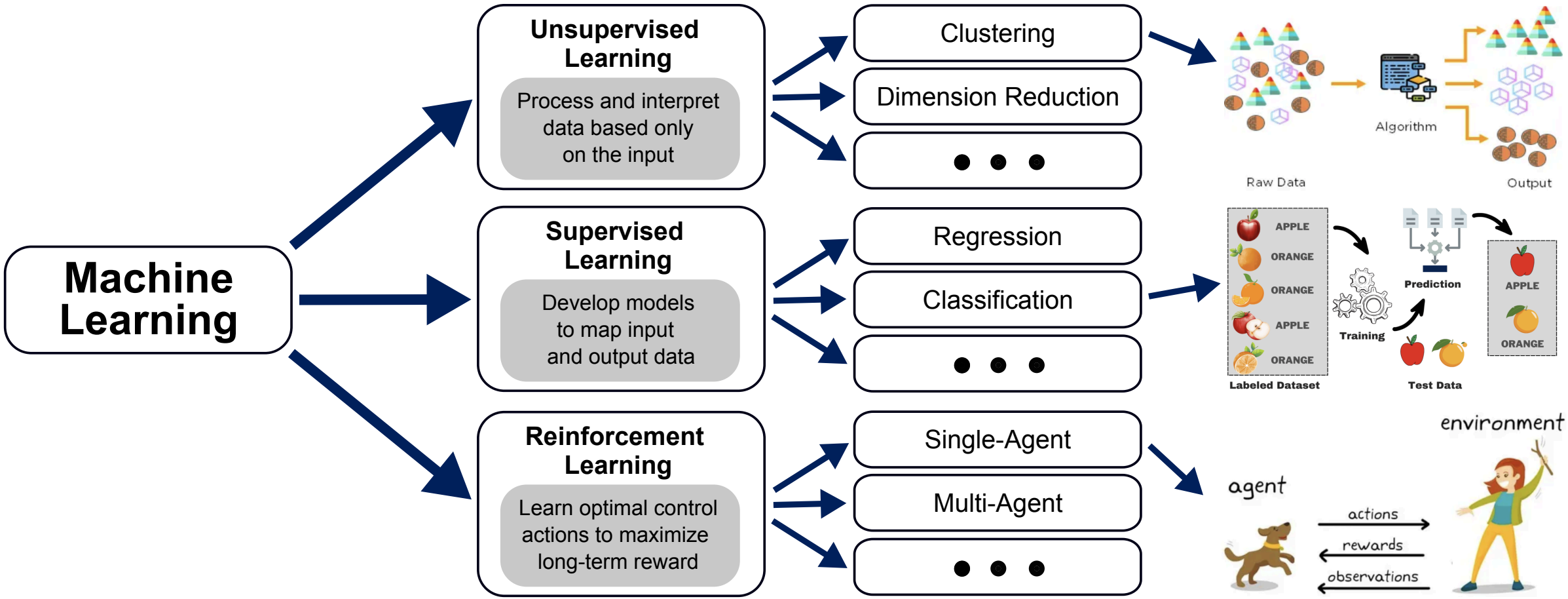
Where are we?

Chapter	Topic	Content
	Basics & tabular methods	
1-5	Bandits, MDPs, Dynamic Programming, Monte Carlo, TD Learning	RL basics in finite dimensions
	Deep-learning-based methods	
6	Brief introduction to deep learning	The basics for what comes next
7	Value function approximation	
8	Deep Q-learning	
9	Policy gradients	
10	Actor-critic algorithms	
11	Advanced algorithms (Part I): From policy gradient to PPO	
12	Advanced algorithms (Part II): From Q-learning to Soft Actor-Critic	
	Model-Based Control	
	Advanced Topics	

Lecture contents

The three main learning paradigms

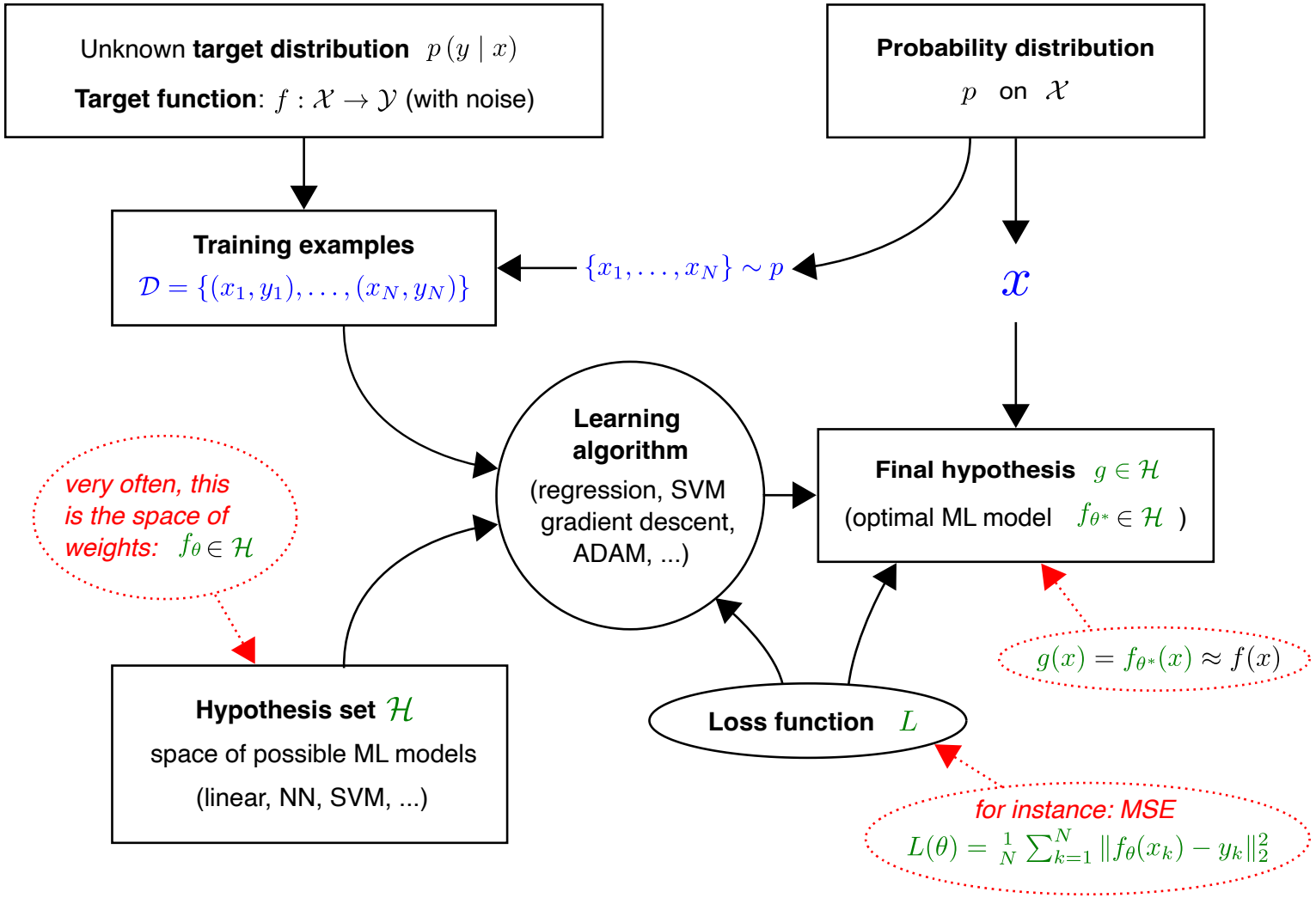
The three main learning paradigms



The three big learning paradigms (Abdelwanis et al. 2026) (Examples: Unsupervised, Supervised, RL)

Supervised learning

The learning diagram



The learning diagram (Abu-Mostafa, Magdon-Ismail, and Lin 2012)

Generalization

- What does it mean to have a perfect model on your training dataset $\mathcal{D}_{\text{train}}$, i.e., $L(\theta; \mathcal{D}_{\text{train}}) = 0$?
  $\Rightarrow$ we have simply memorized the data!
- Learning means that we get **good predictions on unseen data**:

$$\underbrace{L(\theta; \mathcal{D}_{\text{test}})}_{\text{out-of-sample error}} \approx \underbrace{L(\theta; \mathcal{D}_{\text{train}})}_{\text{in-sample error}}.$$

Supervised learning
Given a labeled (and probably noisy) dataset $\mathcal{D}_{\text{train}} = \{(x_1, y_1), \dots, (x_N, y_N)\}$, approximate the unknown mapping $f : \mathcal{X} \to \mathcal{Y}$ by a parametrizable ML model $f_\theta : \mathcal{X} \to \mathcal{Y}$, such that

$$f_\theta(x_k) = \hat{y}_k \approx y_k \quad \forall\, (x_k, y_k) \in \mathcal{D}_{\text{test}}.$$

- **Goodness of fit** can be measured via many different metrics (e.g., mean squared error, classification accuracy, etc.).
- The dimension d of model parameters $\theta \in \mathbb{R}^d$ is adjustable in many model families, which trades off **bias** with **variance** (among other factors, leading to so-called under- and overfitting).

- On top of θ, an ML model might also have **hyperparameters** that can be optimized (e.g., number of layers in a neural network).

Bias-variance tradeoff (1)

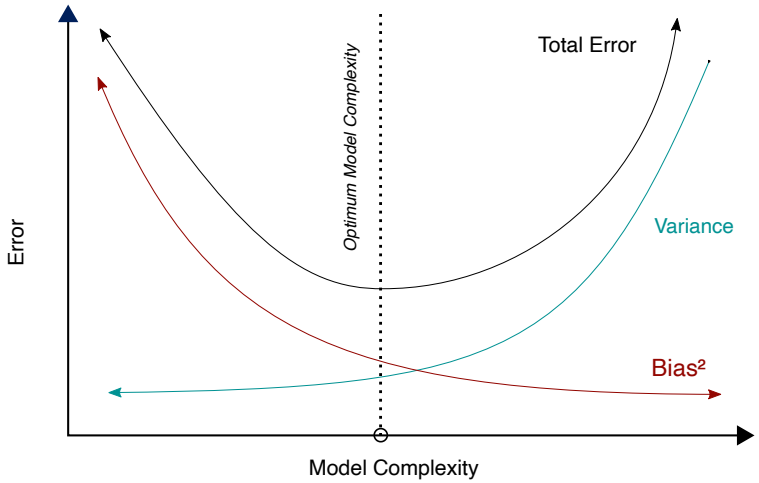
- In the ML context, **bias** denotes the error of the *average* model $\overline{f}_\theta$ when repeating the training with different datasets $\mathcal{D}_{\text{train},1}, \mathcal{D}_{\text{train},2}, \ldots$:

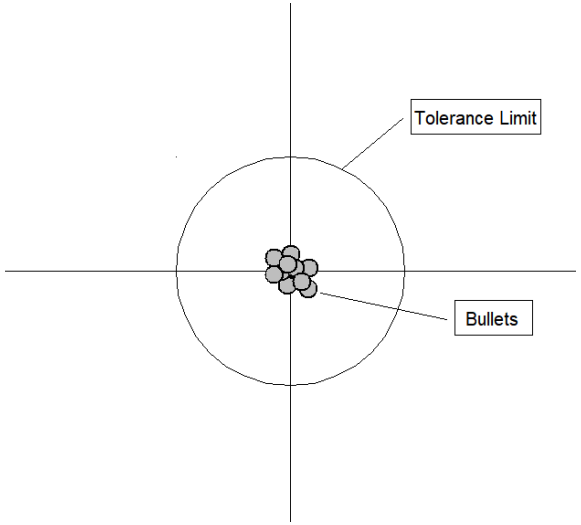
$$\text{bias} = \mathbb{E}_{x \sim \mathcal{D}_{\text{test}}} \left[\left(\overline{f}_\theta(x) - f(x) \right)^2 \right].$$

- **variance** denotes the variability in between the individual training runs:

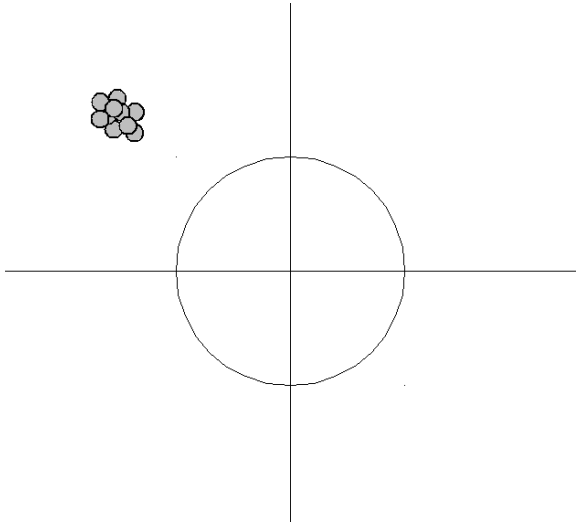
$$\text{variance} = \mathbb{E}_{x \sim \mathcal{D}_{\text{test}}} \left[\mathbb{E}_{\mathcal{D} \sim \{\mathcal{D}_{\text{train},\ell}\}_{\ell=1}^{\infty}} \left[\left(f_\theta^{(\mathcal{D})}(x) - \overline{f}_\theta(x) \right)^2 \right] \right].$$

⇒ Often a matter of model complexity.

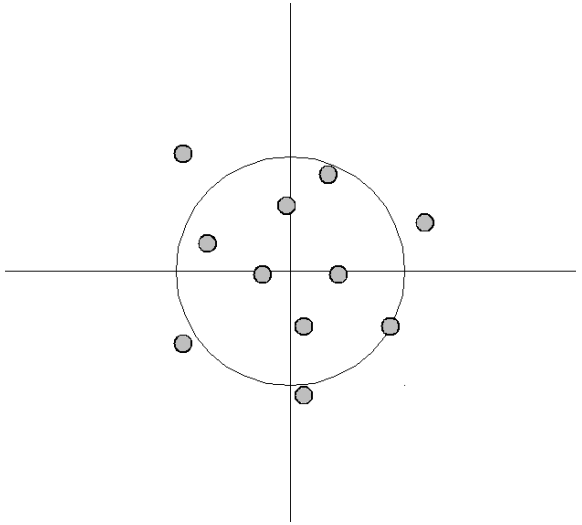




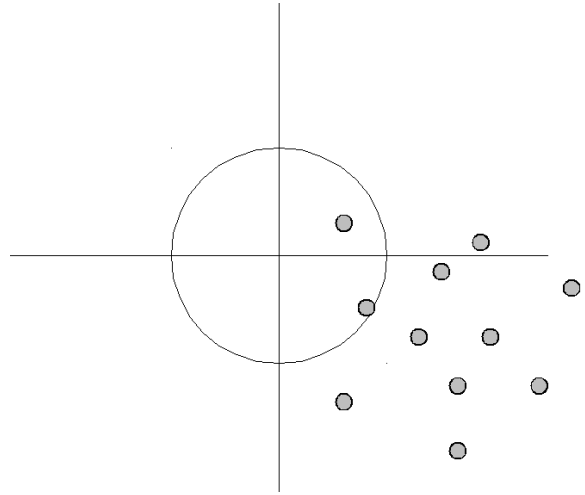
Low bias, low variance



Large bias, low variance



Low bias, large variance



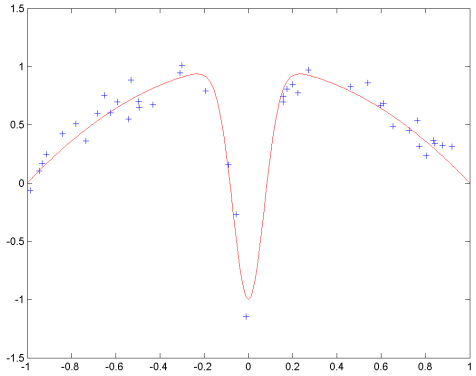
Large bias, large variance [Wikipedia]

Bias-variance tradeoff (2)

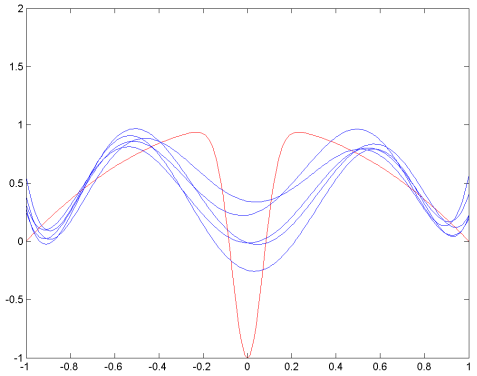
Example (see Wikipedia): Fitting a model of serveral radial basis functions to noisy trainig data:

$$f_\theta(x) = \sum_{k=1}^{d} \theta_k \exp\left(-\frac{1}{2}\frac{x - c_k}{\sigma_k^2}\right).$$

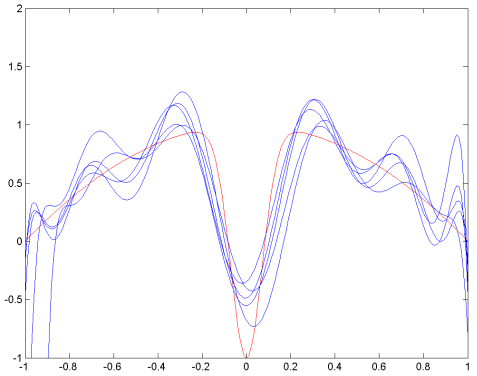
- For a wide spread (i.e., large σ_k), the bias is high: the RBFs cannot fully approximate the function (especially the central dip), but the variance between different trials is low.
- As spread decreases (image 3 and 4) the bias decreases: the blue curves more closely approximate the red…
- … but the variance between trials ($\mathcal{D}_{\text{train},1}$, $\mathcal{D}_{\text{train},2}$, …) increases.



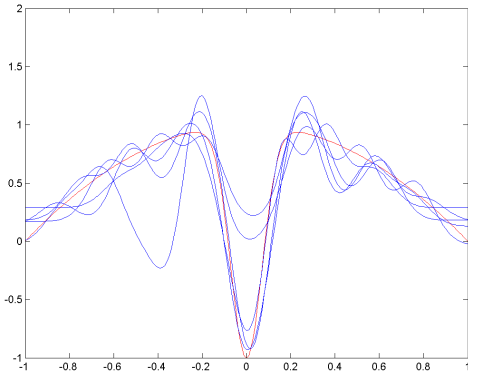
Function and training data $\mathcal{D}_{\text{train},1}$.



Wide spread RBFs.

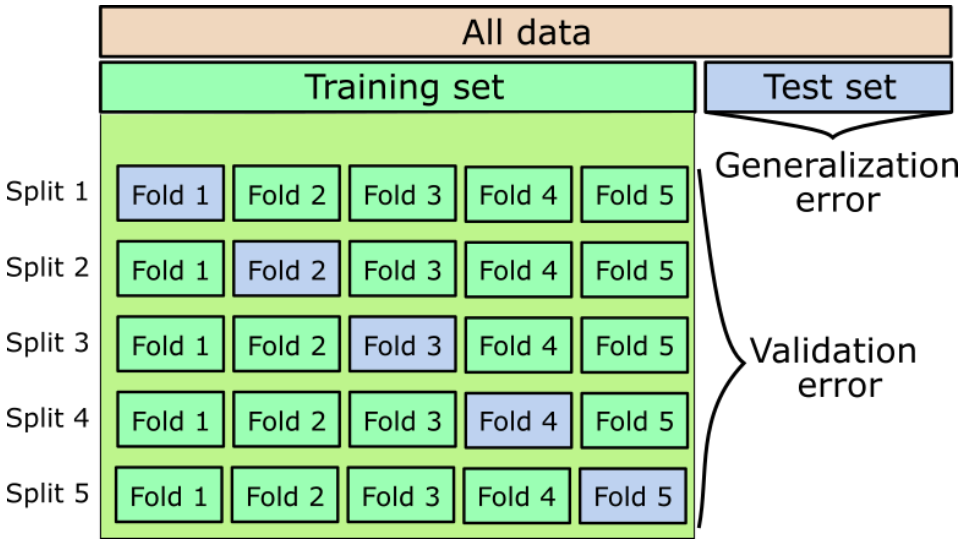


Medium spread RBFs.



Small spread RBFs.

Cross-validation

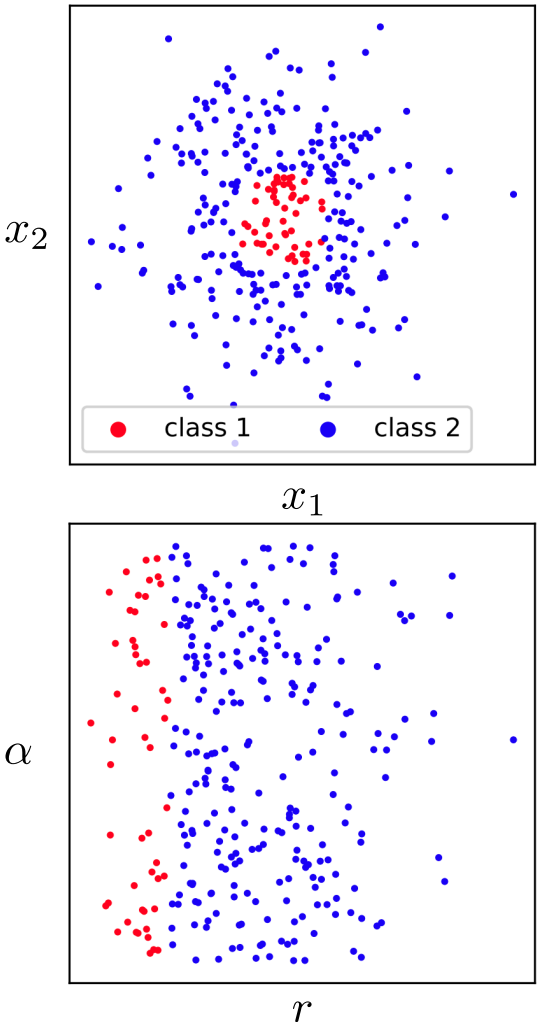


5-fold CV example (Abdelwanis et al. 2026).

- Training is repeated k times with k different splits of the training set.
- Each observation serves as unseen instance (blue boxes) at least once.
- The validation error is an indicator for tuning hyperparameters.
- Example of a k-fold Cross-validation (CV).

Means to improve a supervised learning model

- Collecting **more data**, i.e., increasing N.
- **Reducing noise** in the data.
- **Improving the distribution** within the dataset, i.e., ensuring that the data set is representative of the problem domain.
- **Choosing a more appropriate model**. A genereal rule is to select the model according to the amount of data one has, not according to the expected complexity of the funtion to approximate.
- **Optimizing hyperparameters** of the model.
- **Ensemble learning**: Averaging over several different models.
- **Including knowledge**, e.g., in the form of tailored features (feature engineering) or informed loss functions.
  - This is known as **inductive bias** in the ML literature.



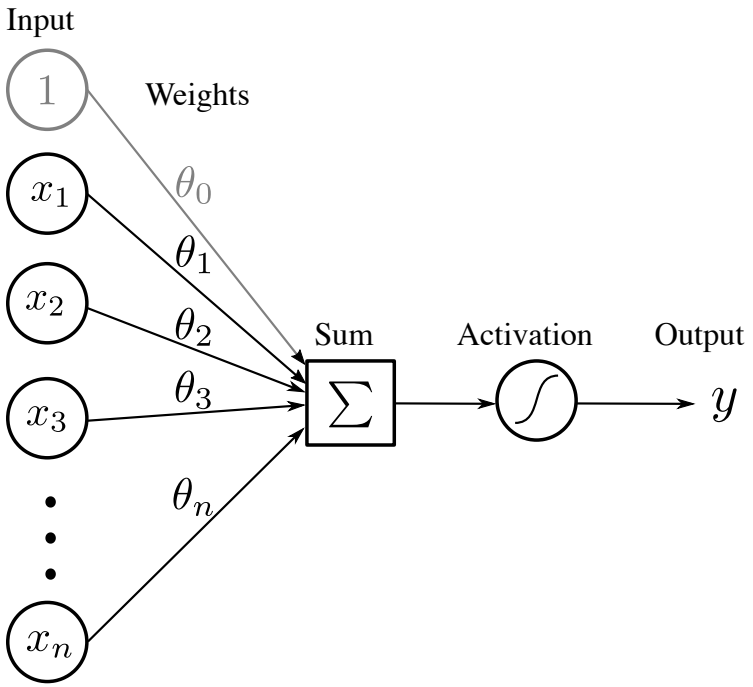
Euclidean vs. polar coordinates/features in binary classification (Abdelwanis et al. 2026).

Deep neural networks

Artificial neural networks

Artificial neural networks (ANNs) are nonlinear function approximators $\hat{y} = f_\theta(x)$ that

- are end-to-end differentiable.
- are stacks of minimal units, the **artificial neurons**.



An artificial neuron (Abdelwanis et al. 2026).

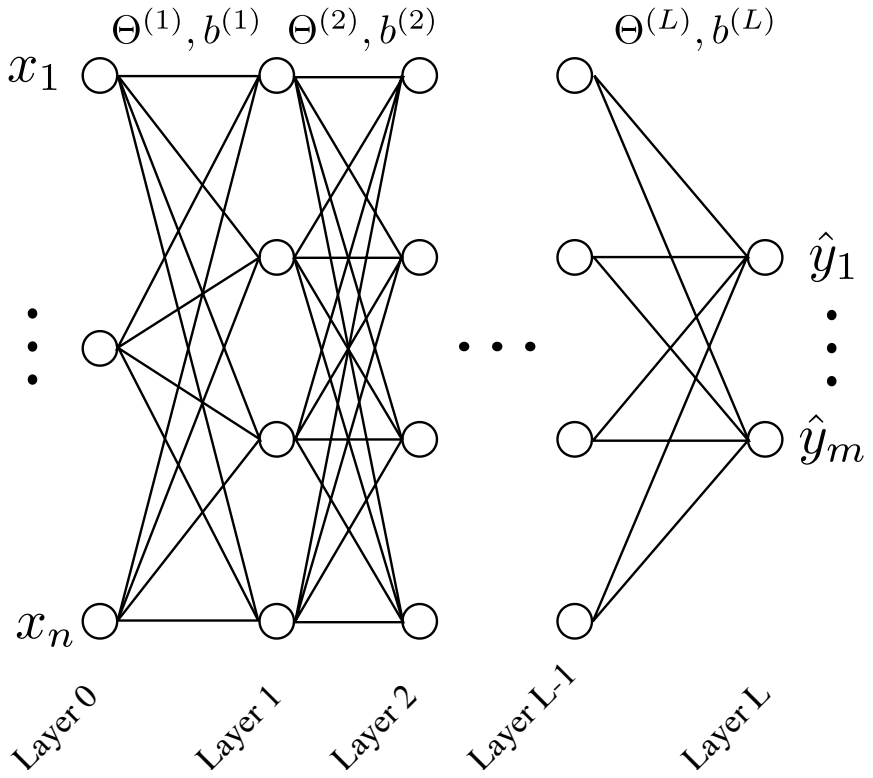
- An ANN consists of **nodes** or **neurons** in one or more layers.
- Each node transforms the weighted sum of all previous nodes (plus a potential **bias term**) through an **activation function** σ:

$$\sigma\left(\theta_0 + \sum_{k=1}^{n} \theta_k x_k\right).$$

- The weighted connections are called **edges**, which represent the ANN's parameters.

Multi-layer perceptron

Standard model of supervised learning: **multi-layer perceptron** or **feed-forward ANN**.



MLP architecture (Abdelwanis et al. 2026).

- Only forward-flowing edges.
- The depth L and width $H^{(\ell)}$ are hyperparameters.
- With $\sigma^{(\ell)}$ and $z^{(\ell)}$ denoting the activation function and **activation** of layer ℓ respectively, we get for the output in the ℓ^{th} layer.

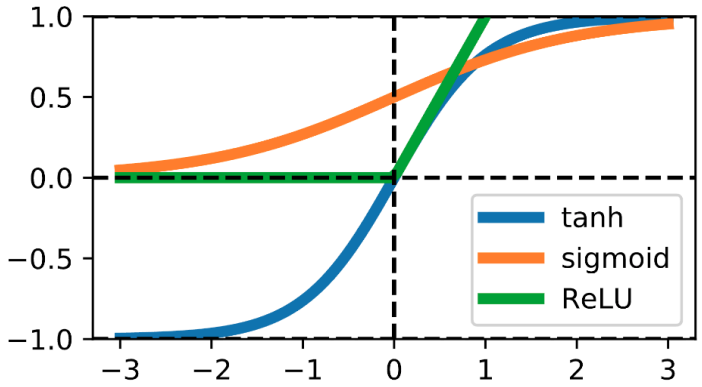
$$x^{(\ell)} = \sigma^{(\ell)}\Big(\underbrace{\Theta^{(\ell)} x^{(\ell-1)} + b^{(\ell)}}_{z^{(\ell)}}\Big),$$

  with input $x^{(0)} = x$ and output $x^{(L)} = y$:

- Training:
  - Summarize the full set of parameters (i.e., weight matrices $\Theta^{(\ell)} \in \mathbb{R}^{H^{(\ell)} \times H^{(\ell-1)}}$ and biases $b^{(\ell)}$) under θ.
  - Iteratively update the weights using gradient information.

Activation functions

- The source of nonlinearity in neural networks.*
- Common choices for $\sigma(z)$ are
  - $\sigma(z) = \tanh(z)$,
  - Sigmoid: $\sigma(z) = \frac{1}{1+e^{-z}}$,
  - Rectified linear unit (ReLU): $\sigma(z) = \max(0, z)$,



Exemplary activation functions (Abdelwanis et al. 2026).

- The activation of the output layer, $\sigma^{(L)}(z)$, is task-dependent. For instance,
  - regression: $y = \sigma^{(L)}(z^{(L)}) = z^{(L)}$, i.e., $\sigma^{(L)} = \mathbf{Id}$ is the identity mapping.
  - binary classification: sigmoid (i.e., probability), followed by a rounding step to either 0 or 1.
  - multi-class classification: $y_i = \frac{\exp(z^{(L)}{}_i)}{\sum_j \exp(z^{(L)}{}_j)}$ (softmax).

* Without nonlinear activation functions, every ANN collapses to a single matrix-vector multiplication $y = \hat{\Theta}x$:

$$x^{(\ell+2)} = \Theta^{(\ell+2)} x^{(\ell+1)} = \Theta^{(\ell+2)} \left(\Theta^{(\ell+1)} x^{(\ell)}\right) = \left(\Theta^{(\ell+2)} \Theta^{(\ell+1)}\right) x^{(\ell)} = \hat{\Theta} x^{(\ell)}.$$

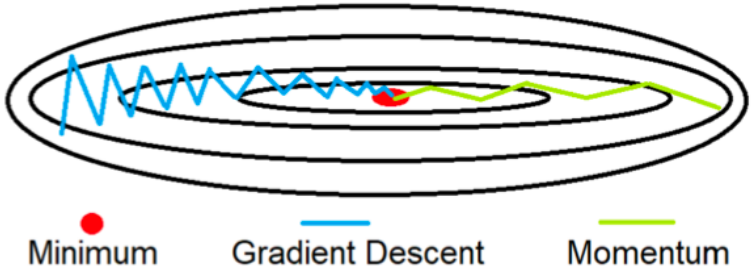
For non-zero biases, we obtain an *affine* transformation instead.

Training (1)

- Training is performed in an iterative manner:

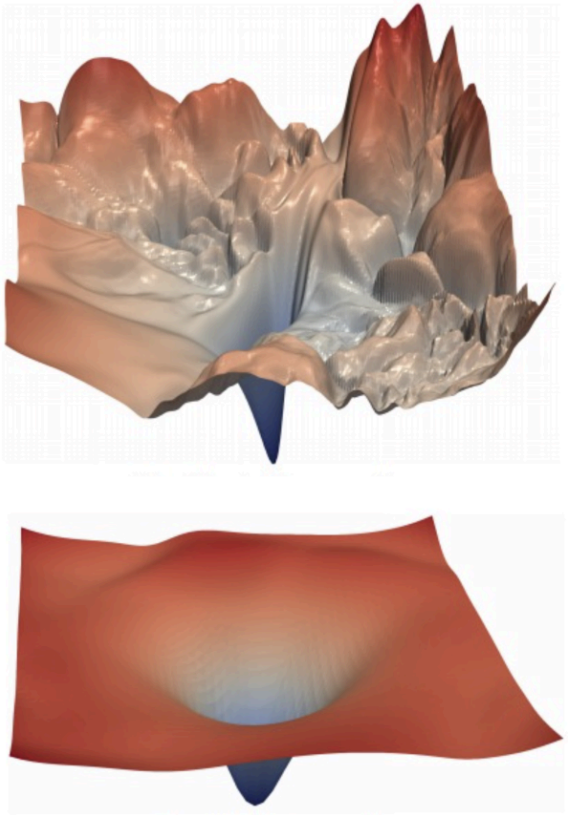
$$\theta \leftarrow \theta + \eta\delta\theta.$$

  - $\eta \in \mathbb{R}_{>0}$ is the **step size** or **learning rate**.
  - $\delta\theta \in \mathbb{R}^d$ is the **update direction**, usually a gradient-based **descent direction**.
  - Numerous variants for $\delta\theta$. The strongest contain additional **momentum** terms such as the *Adam* algorithm (Kingma and Ba 2014). We remember the previous update $\delta\theta^-$ and thereby flatten out zig-zag behavior.



Source

- First, we need to define a **loss function** that we wish to minimize.
  - Regression: (root) mean square error, mean absolute error.
  - Classification: cross entropy.
  - Additional terms, e.g., regularization, physics information, …
- Iterations over the dataset $\mathcal{D}_{\text{train}}$ are called **epochs**.



The loss landscape of deep neural networks (with and without skip-connection) (Li et al. 2018).

Training (2)

- The descent direction is computed by taking the derivative of the loss function w.r.t. the weight vector. In terms of the MSE:

$$\begin{aligned}\nabla L(\theta) &= \nabla \left(\frac{1}{N} \sum_{k=1}^{N} \| f_\theta(x_k) - y_k \|_2^2 \right) \\ &= \frac{1}{N} \sum_{k=1}^{N} \nabla \| f_\theta(x_k) - y_k \|_2^2 && \text{(Linearity of the sum)} \\ &= \frac{1}{N} \sum_{k=1}^{N} (2(f_\theta(x_k) - y_k) \nabla f_\theta(x_k)) && \text{(Chain rule of differentiation)}\end{aligned}$$

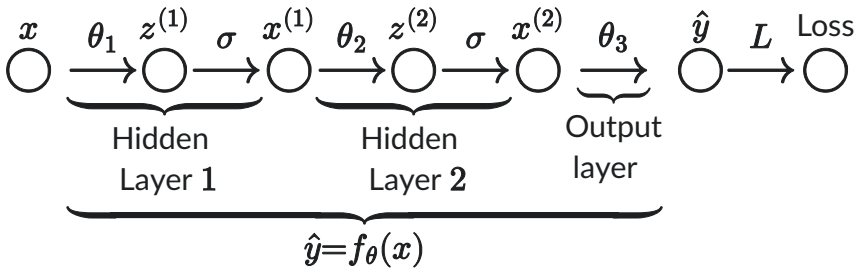
- This means that we need to *propagate* the error through our model f_θ.
- Since a neural network is a chain of neurons, we need to apply the **chain rule** of differentiation over the layers.
- As a consequence the loss is **backpropgated** through the network to determine the individual descent directions: $\frac{\partial L}{\partial \theta_i}$.

This is called the **backpropagation** algorithm. We require one forward pass and one backward pass for every data tuple (x_k, y_k). Taking the average gives us the average steepest-descent improvement over the dataset $\mathcal{D}_{\text{train}}$ for the current θ.

Backpropagation example

Let's consider a very simple network with $x, y \in \mathbb{R}$ and two hidden layers with a single neuron each.

Symbolic



In mathematical terms

$$\hat{y} = \theta_3\left(x^{(2)}\right) = \theta_3\left(\sigma\left(z^{(2)}\right)\right) = \theta_3\left(\sigma\left(\theta_2\left(x^{(1)}\right)\right)\right)$$

$$= \theta_3\left(\sigma\left(\theta_2\left(\theta\left(z^{(1)}\right)\right)\right)\right) = \theta_3 \underbrace{\sigma\left(\theta_2 \overbrace{\sigma\left(\theta_1 x\right)}^{\text{Hidden Layer 1}}\right)}_{\text{Hidden Layer 2}}$$

Let's assume a single sample (x, y) and the MSE loss: $L(\theta) = (\hat{y} - y)^2$. Using the chain rule, we get:

1. Gradient w.r.t. θ_3:

$$\frac{\partial L}{\partial \theta_3} = \underbrace{\frac{\partial L}{\partial \hat{y}}}_{=2(\hat{y}-y)} \frac{\partial \hat{y}}{\partial \theta_3}.$$

2. Gradient w.r.t. θ_2:

$$\frac{\partial L}{\partial \theta_2} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial x^{(2)}} \underbrace{\frac{\partial x^{(2)}}{\partial z^{(2)}}}_{=\sigma'} \frac{\partial z^{(2)}}{\partial \theta_2}.$$

3. Gradient w.r.t. θ_1:

$$\frac{\partial L}{\partial \theta_1} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial x^{(2)}} \underbrace{\frac{\partial x^{(2)}}{\partial z^{(2)}}}_{=\sigma'} \frac{\partial z^{(2)}}{\partial x^{(1)}} \underbrace{\frac{\partial x^{(1)}}{\partial z^{(1)}}}_{=\sigma'} \frac{\partial z^{(1)}}{\partial \theta_1}.$$

$\Rightarrow$ we propagate the loss **back** through the network, reusing most of the previous calculations!

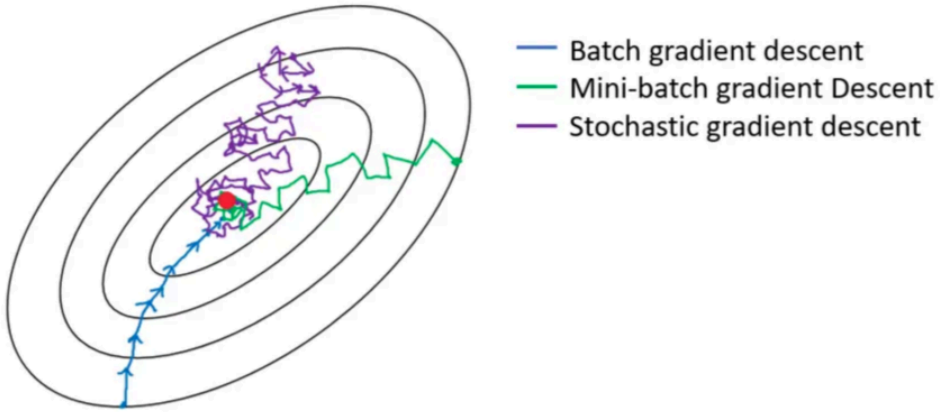
Stochastic gradient descent and batch learning

- The cost for a single gradient step scales with the dataset size N, which can be **very large**.
- For each sample, we need to perform one forward and one backward pass.
- **Stochastic gradient descent** (**SGD**): massive speedup by using a single sample per step insetad of the entire dataset,

$$\delta\theta = \nabla L(\theta; x_k), \qquad k \sim U(\{1, \dots, N\}).$$

- **Mini-batch gradient descent** with $s \in \{1, \dots, N\}$ samples per step: compromise ground between efficiency and noisiness,

$$\delta\theta = \frac{1}{s} \sum_{k \in \mathcal{J}} \nabla L(\theta; x_k), \qquad \text{where } \mathcal{J} \text{ is an } s\text{-dimensional, ranodmly drawn subset of } \{1, \dots, N\}.$$



Gradient descent vs. SGD vs. Mini-batch gradient descent [Sourcre].

Regularization

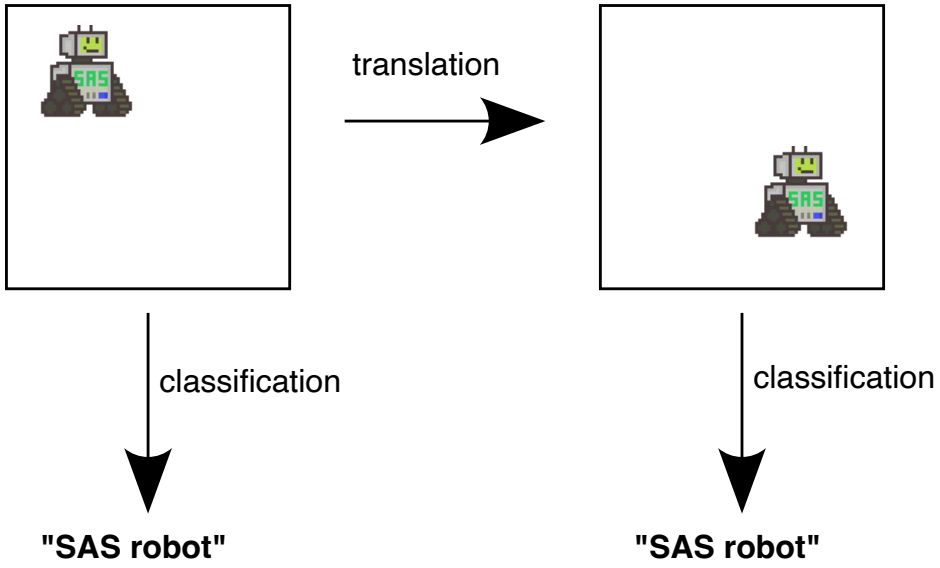
In order to mitigate overfitting (i.e., good performacne on $\mathcal{D}_{\text{train}}$, poor generalization), neural networks can be regularized by

- **Weight decay**: adding an ℓ_2 penalty term to the weights: $L(\theta) + \lambda \|\theta\|_2^2$
- **Layer normalization** during training: all layers' activations are normalized separately by standard scaling,
- **Dropout**: randomly disable nodes' contribution.
  - This helps especially in deep networks,
  - and effectively builds an ensemble of ANNs with shared edges.

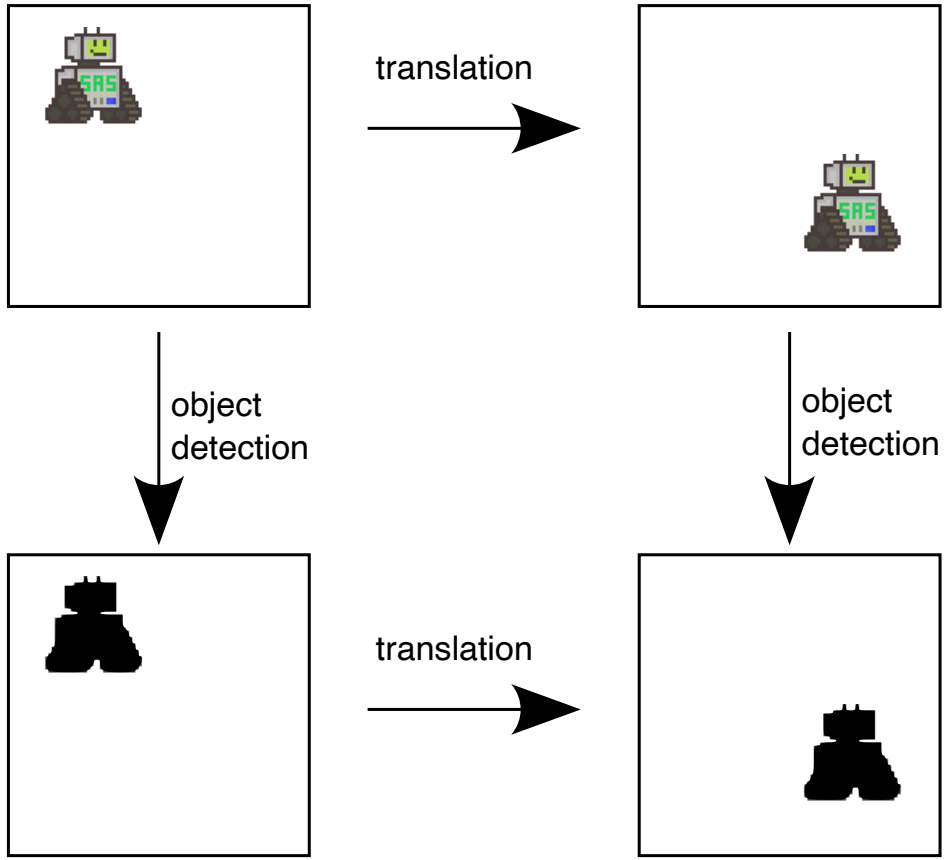
Convolutional neural networks

In machine learning (as well as in nature) **many tasks are independent of absolute position.**

A function is **invariant** under some transformation if it's output y remains unchanged under transformations of the input x.



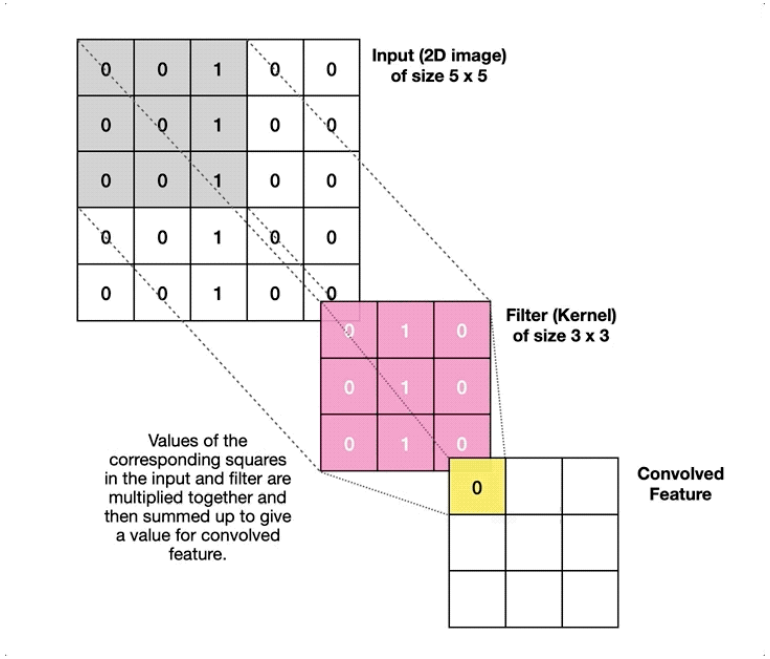
A function is **equivariant** under some transformation if it's output y is transformed "in the same way" as the input x.



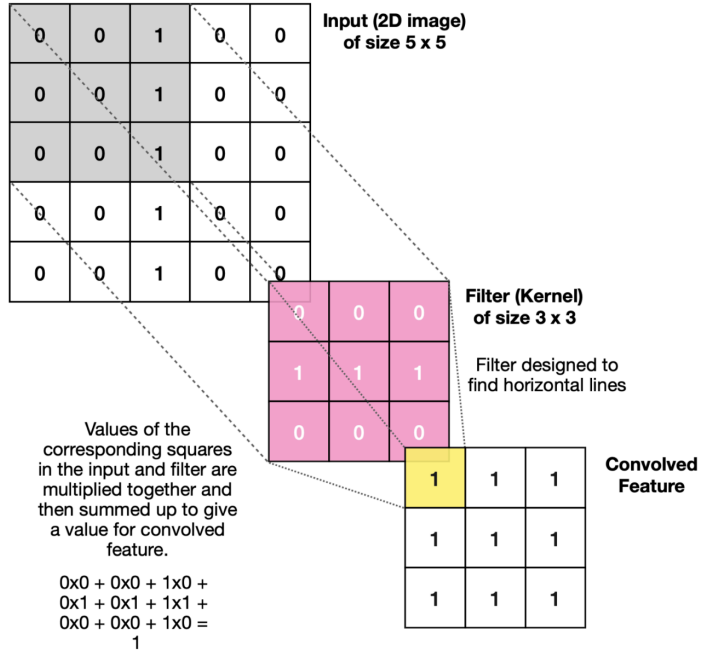
💡 These ideas can be formalized and extended under the framework of *group theory*, see also *geometric deep learning* (Bronstein et al. 2021).

Weight sharing in CNNs

- What does this mean for learning?
  ⇒ We have to learn the same thing in different locations!
  ⇒ An “edge” remains an “edge”, no matter where we are.
- Instead of training a fully connected layer, we train the weights of a **kernel** that moves over the input
- Same weights in every location ⇒ weight sharing!



- Architecture is closely related to fully connected NNs:
  - Inner product between kernel weights θ and input x ⇒ activation z.
  - Multiple kernels ⇒ multiple **channels**.
  - Then comes a nonlinear activation function $\sigma(z)$.
  - Further followed by additional steps (see next slide).

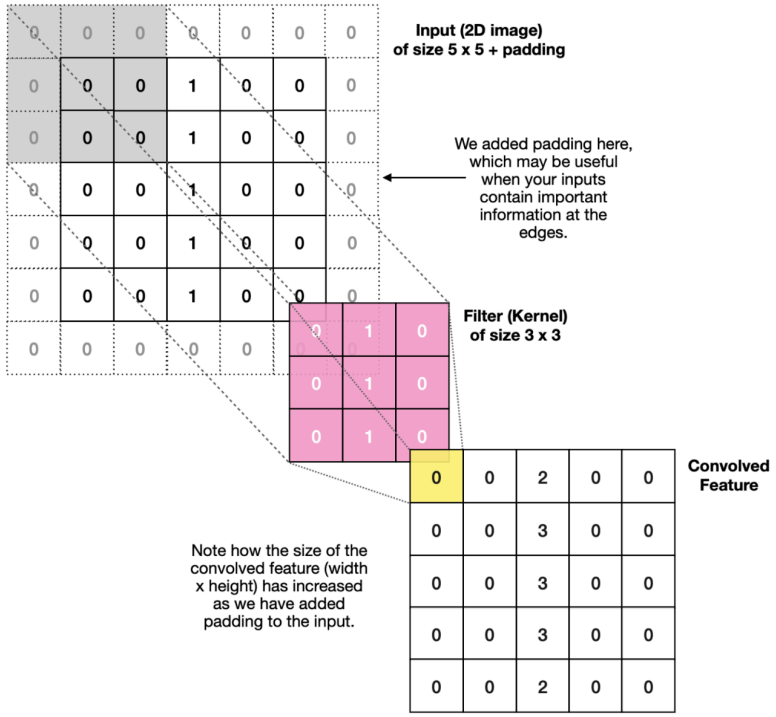


A kernel sweeping over an input [Source]

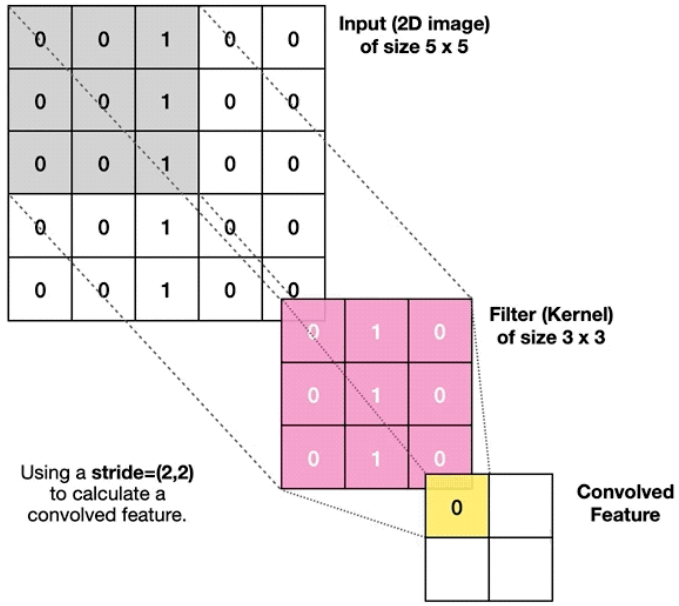
Second kernel = second channel [Source]

Ingredients of a CNN

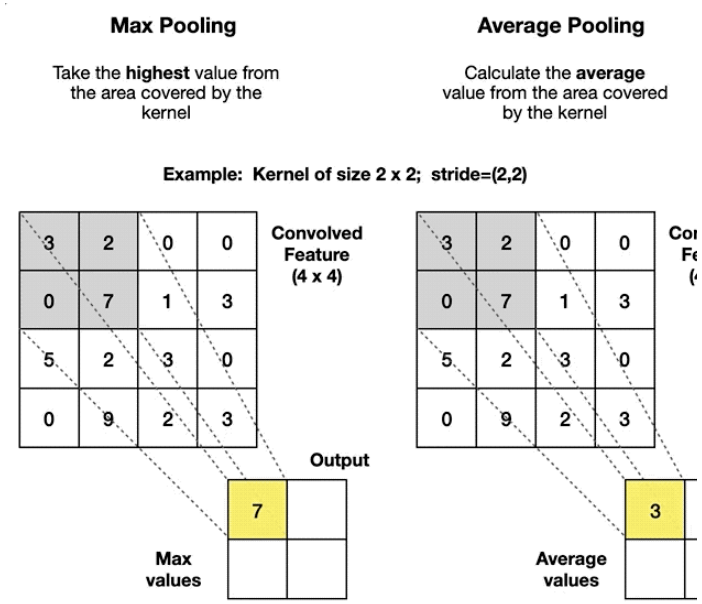
Further considerations (**padding** & **stride**) and additional steps (e.g., **pooling**)



Padding [Source]

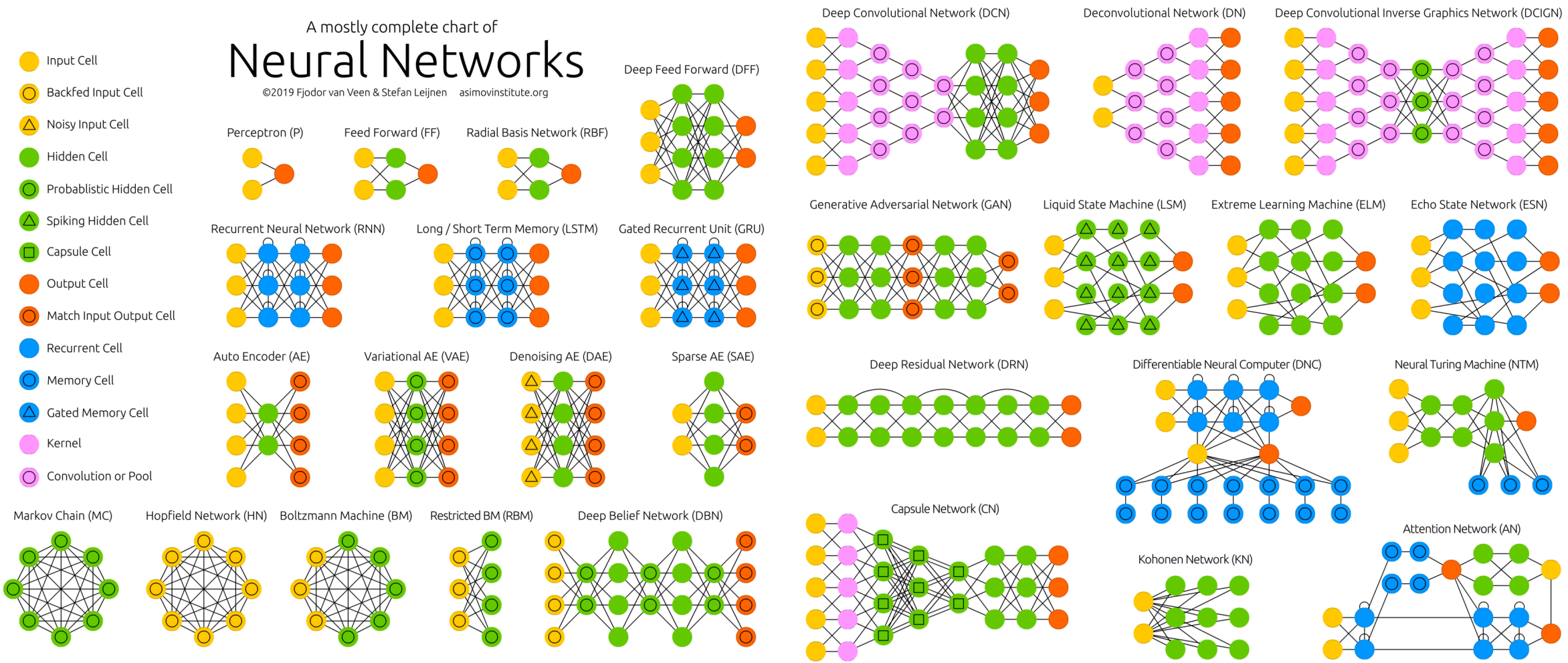


Stride [Source]



Pooling [Source]

Neural network architectures



The neural network zoo [Asimov Institute].

Linear regression as a special case

The linear model: single-layer NN with $\sigma = \mathrm{Id}$

- Let's assume that we only have a single layer directly mapping inputs $x \in \mathbb{R}^n$ to outputs $y \in \mathbb{R}^m$:

$$\hat{y} = f_\Theta(x) = \Theta^\top x \qquad \text{with} \quad \Theta \in \mathbb{R}^{m \times n}.$$

- Let's consider the usual MSE loss:

$$L(\Theta) = \frac{1}{N} \sum_{k=1}^{N} \left\| \Theta^\top x_k - y_k \right\|_2^2.$$

- We organize the data in a slightly different fashion, i.e.,

$$X = \begin{pmatrix} - & x_1^\top & - \\ & \vdots & \\ - & x_N^\top & - \end{pmatrix} \in \mathbb{R}^{N \times n}, \qquad Y = \begin{pmatrix} - & y_1^\top & - \\ & \vdots & \\ - & y_N^\top & - \end{pmatrix} \in \mathbb{R}^{N \times m}.$$

- Then the model can predict all N samples at the same time by a matrix-matrix product: $\hat{Y} = X\Theta$.

- The associated loss function then simply is the **Frobenius norm** over the dataset $\mathcal{D}_{\text{train}} = (X, Y)$:

$$L(\Theta; \mathcal{D}_{\text{train}}) = \frac{1}{N} \|X\Theta - Y\|_F^2 = \frac{1}{N} (X\Theta - Y)^\top (X\Theta - Y).$$

Solving the regression problem

- What's the necessary condition for the minimizer of a function?
  $\Rightarrow$ The gradient has to be zero!

$$\frac{\partial L(\Theta; \mathcal{D}_{\text{train}})}{\partial \Theta} = \frac{\partial}{\partial \Theta}\left[\frac{1}{N}(X\Theta - Y)^\top (X\Theta - Y)\right] = \frac{2}{N} X^\top (X\Theta - Y)$$

- Optimal solution:

$$\begin{aligned} \frac{2}{N} X^\top (X\Theta^* - Y) &\stackrel{!}{=} 0 \\ \Leftrightarrow \quad X^\top X\Theta^* &= X^\top Y \\ \Leftrightarrow \quad \Theta^* &= (X^\top X)^{-1} X^\top Y = X^\dagger Y, \end{aligned}$$

  where $X^\dagger$ is the **pseudo-inverse** of X.

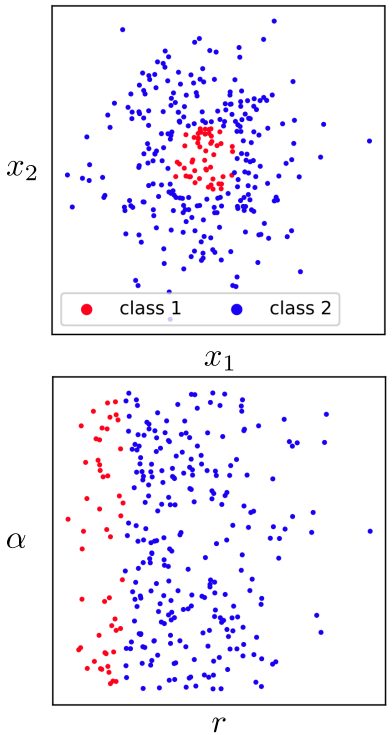
- Training is replaced by a simple one-step learning approach (using, e.g. `numpy.linalg.pinv`).
- Since the loss function L is convex, the solution Θ^* is not a local, but the **global optimum**.

Features as an additional pre-processing step

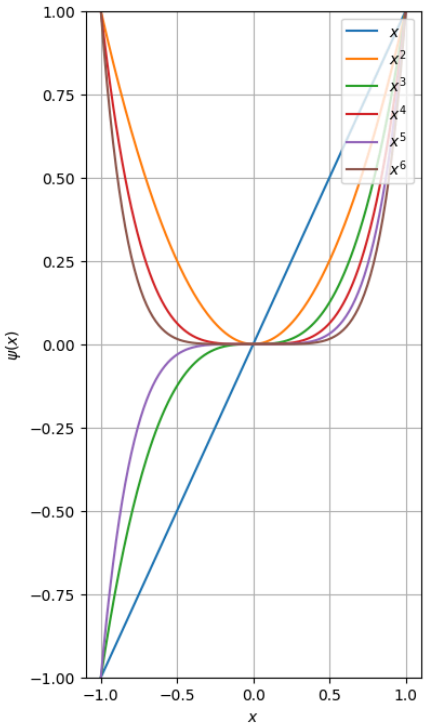
- If we introduce *feature functions* $\psi_i : \mathbb{R}^n \to \mathbb{R}$, $\psi_i(x) = z_i$, then we can significantly improve the performance:

$$\psi(x) = [\psi_1(x), \dots, \psi_q(x)]^\top.$$

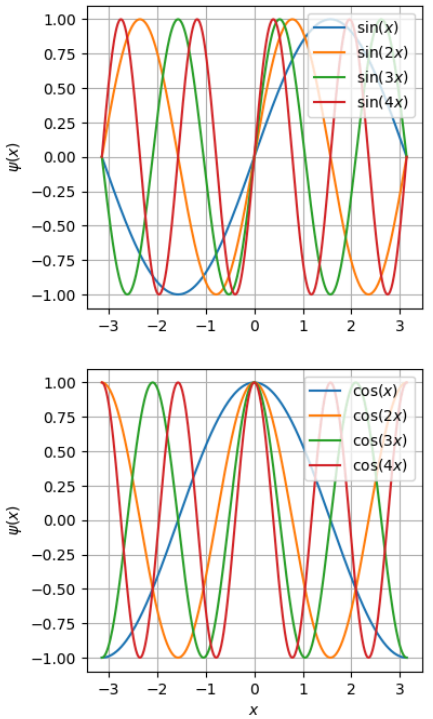
- Different ways of introducing features:



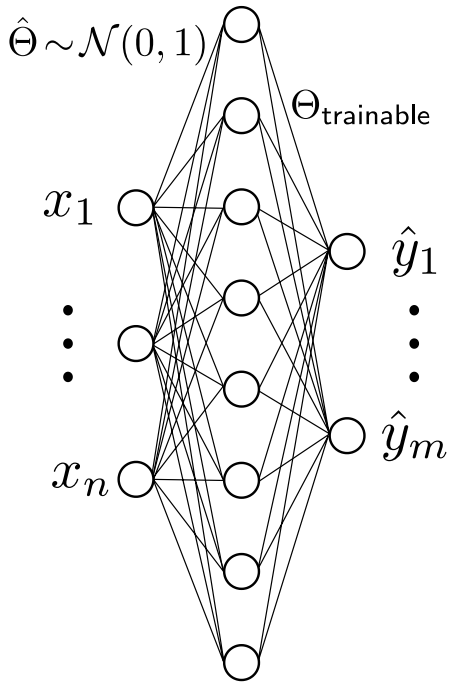
Tailored (Abdelwanis et al. 2026)



Manual: Polynomials



Manual: Fourier modes



Randomly: Extreme Leraning Machine

💡 The layers 1 to $L-1$ of a neural network can also be seen as a feature transform, **learned autmoatically** from data.

Summary / what you have learned

Summary / what you have learned

- There are **three main learning paradigms**:
  - **supervised learning** for function approximation of input-to-output mappings,
  - **unsupervised learning** for clustering, pattern detection or dimensionality reduction,
  - **reinforcement learning** for sequential decision making in dynamic envionments.
- In supervised learning, we aim to approximate an unknown function f by a parametric function f_θ by adjusting θ such that a loss function L is minimized over a training dataset $\mathcal{D}_{\text{train}}$.
  - The central goal is **generalization** beyond the training data.
  - We have to handle the **bias-variance-tradeoff**, which is often a matter of model complexity.
  - There are many techniques to improve learning such as cross-validation, regularization, inductive biases, ...
- **Deep neural networks** come in a great range of varieties.
  - They are a large network of a single unit known as the perceptron.
  - The general structure consists of linear transformations, followed by nonlinear activation functions.
  - Training is realized via **backpropagation** and **gradient descent**.
  - Various versions for the descent direction, for instance using **momentum**.
  - Improved efficiency (at the cost of noise) by using **stochastic gradient descent** or **mini-batch gradient descent**.
- Linear regression is the special case of a single linear layer.
  - Training can be realized in closed form using the **pseudo-inverse**.

References

Abdelwanis, Ali, Barnabas Haucke-Korber, Darius Jakobeit, Wilhelm Kirchgässner, Marvin Meyer, Maximilian Schenke, Hendrik Vater, Oliver Wallscheid, and Daniel Weber. 2026. “Reinforcement Learning: A Comprehensive Open-Source Course.” *Journal of Open Source Education* 9 (97). The Open Journal: 306. doi:10.21105/jose.00306.

Abu-Mostafa, Yaser S, Malik Magdon-Ismail, and Hsuan-Tien Lin. 2012. *Learning from Data*. Vol. 4. AMLBook New York.

Bengio, Yoshua, Ian Goodfellow, Aaron Courville, et al. 2017. *Deep Learning*. Vol. 1. MIT press Cambridge, MA, USA.

Bishop, Christopher M. 2006. *Pattern Recognition and Machine Learning*. Vol. 4. 4. Springer.

Bronstein, Michael M., Joan Bruna, Taco Cohen, and Petar Veličković. 2021. “Geometric Deep Learning: Grids, Groups, Graphs, Geodesics, and Gauges.” *arXiv:2104.13478*, April.

Hastie, Trevor. 2009. “The Elements of Statistical Learning: Data Mining, Inference, and Prediction.” springer.

Kingma, Diederik P, and Jimmy Ba. 2014. “Adam: A Method for Stochastic Optimization.” *arXiv Preprint arXiv:1412.6980*.

Li, Hao, Zheng Xu, Gavin Taylor, Christoph Studer, and Tom Goldstein. 2018. “Visualizing the Loss Landscape of Neural Nets.” *Advances in Neural Information Processing Systems* 31.